🎓 MACHINE LEARNING FUNDAMENTALS

ASSIGNMENT NO. 02



📘 Assignment Title Page

SUBMITTED BY

Abdul Sami

Registration Number: FA24-BSE-124

SECTION: A

COURSE INFORMATION

Field	Details
Course Title	Machine Learning Fundamentals
Instructor	Dr. Rao Adeel Nawab
Assignment	Assignment No. 02
Student Name	Abdul Sami
Registration No.	FA24-BSe-124

Department of Computer Science

⬇️ END OF TITLE PAGE ⬇️

CARDIOLOGY AI PIPELINE

❤️ Heart Disease Prediction Model

An End-to-End Machine Learning Workflow, Clinical Validation & Deployment Pipeline

---

🔗 Project Applications & Repository

🚢 Titanic Streamlit App

❤️ Heart Disease Streamlit App

💻 GitHub Repository

PLEASE CLICK ON THE ICONS ABOVE TO GET TO YOUR REQUIRED LINK.THANK YOU

📋 Table of Contents

Step	Stage	Description
Step 1	Import Libraries	Import the required Python libraries.
Step 2	Load Sample Data	Load and display the CSV dataset.
Step 3	Understand and Preprocess Sample Data	Inspect and prepare the dataset. Step 3.1: Understand Sample Data Step 3.2: Process Sample Data (Already Completed)
Step 4	Feature Extraction (Already Completed)	Identify input features and target labels.
Step 5	Label Encoding the Sample Data	Skipped / Replaced by Feature Scaling.
Step 6	Execute the Training Phase	Split the data and train the SVC model. Step 6.1: Split Training and Testing Data Step 6.2: Split Training Inputs and Target Labels Step 6.3: Train the Support Vector Classifier (SVC) Step 6.4: Save the Trained Model and Scaler Artifacts
Step 7	Execute the Testing Phase	Test predictions and evaluate performance. Step 7.1: Split Testing Inputs and Target Labels Step 7.2: Load the Saved Model Step 7.3: Evaluate the Machine Learning Model Step 7.3.1: Make Predictions on Testing Data Step 7.4: Calculate Accuracy and Export Predictions Step 7.5: Classification Report and Confusion Matrix
Step 8	Execute the Application Phase	Generate predictions for new user input. Step 8.1: Take Input from User Step 8.2: Convert Input into Feature Vector Step 8.3: Scale the Feature Vector Step 8.4: Load the Saved Model and Scaler Step 8.5: Model Prediction Step 8.5.1: Predict on Scaled Unseen Data
Step 9	Execute the Feedback Phase	Collect feedback and plan improvements. Step 9.1: Collect User and Domain Expert Feedback Step 9.2: Potential Model and Workflow Improvements Step 9.3: Next Version Retraining Strategy

⚡ CODE IMPLEMENTATION STARTS BELOW

Step 1: Import Libraries

- **Objective:** Load all necessary Python libraries required for data manipulation, numerical processing, machine learning modeling, visualization, and artifact serialization.
- **Key Modules:**
  - `pandas` & `numpy` : Data manipulation, vector operations, and tabular data structures.

- `scikit-learn`: Data splitting (`train_test_split`), feature scaling (`StandardScaler`), modeling (`SVC`), and evaluation metrics (`classification_report`, `confusion_matrix`).
- `matplotlib` & `seaborn`: Graphical visualization of performance matrices.
- `pickle`: Model and scaler persistence for production deployment..

```
# Step 1: Import all necessary libraries
import numpy as np
import pandas as pd
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from prettytable import PrettyTable
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import svm
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Step 2: Load Sample Data

Objective: Read the heart disease clinical dataset (`heart.csv`) into a Pandas DataFrame.

Key Operations:

- Inspect row counts, column structure, and statistical summaries.
- **Target Column:** `target` (0 = No Heart Disease, 1 = Heart Disease).
- **Selected Clinical Features:**
  - `cp` — Chest Pain Type
  - `chol` — Cholesterol
  - `thalach` — Maximum Heart Rate
  - `thal` — Thalassemia

```
"""
Purpose of this section:
-------------------------
This part of the code loads the heart disease dataset from a CSV file into a
pandas DataFrame. It then displays the dataset so that students can
see the raw data being used for further processing and model training.
"""

# Read the CSV file and load it into a pandas DataFrame
# 'heart.csv' should be present in the working directory
heart_data = pd.read_csv("heart.csv")

# Print a header to indicate that sample data is being displayed
print("\n\nSample Data:")
print("============\n")

# Configure pandas to display all rows and columns (so nothing gets hidden with '...')
pd.set_option("display.max_rows", None, "display.max_columns", None)

# Print the complete dataset
print(heart_data)
```

Sample Data:
============

	cp	chol	thalach	thal	target
0	0	212	168	3	0
1	0	203	155	3	0
2	0	174	125	3	0
3	0	203	161	3	0
4	0	294	106	2	0
5	0	248	122	2	1
6	0	318	140	1	0
7	0	289	145	3	0
8	0	249	144	3	0
9	0	286	116	2	0
10	0	149	125	2	1
11	0	341	136	3	0
12	1	210	192	2	1
13	0	298	122	3	0
14	0	204	156	0	0
15	1	210	192	2	1
16	2	308	142	2	1
17	0	266	109	3	0
18	1	244	162	2	1
19	2	211	165	2	1
20	2	185	155	2	0
21	0	223	142	2	1
22	0	208	148	2	1
23	2	252	172	2	1
24	2	209	173	2	1
25	0	307	146	3	0
26	2	233	179	2	1
27	1	319	152	2	0
28	2	256	142	1	0
29	0	327	117	2	0
30	0	169	144	1	0
31	1	244	162	2	1
32	0	131	115	3	0
33	2	269	112	3	0
34	2	196	163	2	1
35	2	231	147	2	0
36	3	213	125	2	1
37	0	271	182	2	1
38	0	263	105	3	1
39	2	229	150	3	0
40	2	360	151	2	1
41	2	258	147	3	1
42	0	330	169	2	0
43	0	249	144	3	0
44	1	342	166	2	1
45	0	226	178	2	1
46	1	203	132	1	1
47	0	228	165	3	0
48	2	278	152	2	1
49	0	230	160	3	0
50	3	283	162	2	1

51	0	241	123	3	0
52	2	175	173	2	1
53	2	188	139	3	0
54	0	217	111	3	0
55	0	217	111	3	0
56	3	193	162	3	1
57	1	245	180	2	1
58	2	212	150	3	0
59	1	232	164	2	0
60	1	204	202	2	1
61	2	278	152	2	1
62	0	299	125	2	0
63	2	212	157	2	1
64	1	204	202	2	1
65	3	288	159	3	0
66	2	197	152	2	1
67	0	315	125	1	0
68	2	215	170	2	1
69	0	164	145	3	0
70	0	326	140	3	0
71	0	207	138	3	0
72	0	249	144	2	0
73	0	177	162	3	0
74	0	256	150	3	0
75	2	257	156	2	1
76	2	255	175	2	1
77	0	187	144	3	0
78	1	201	158	2	1
79	1	201	158	2	1
80	2	233	163	3	0
81	2	149	126	2	0
82	2	231	147	2	0
83	2	175	173	2	1
84	2	215	170	2	1
85	1	220	170	2	1
86	2	211	165	2	1
87	0	249	143	2	0
88	0	268	160	2	0
89	0	193	141	3	0
90	2	267	167	2	1
91	0	209	163	2	1
92	0	187	144	3	0
93	0	169	144	1	0
94	1	208	140	2	1
95	0	236	152	2	1
96	0	303	159	2	1
97	0	282	95	3	0
98	0	248	158	1	0
99	2	197	116	2	1
100	2	213	165	2	1
101	2	126	173	3	1
102	1	309	156	3	1
103	2	257	156	2	1
104	3	186	190	1	1
105	0	275	118	2	0
106	0	299	173	3	0

107	1	281	103	3	0
108	0	223	181	3	0
109	0	206	108	2	0
110	0	197	177	2	0
111	0	226	111	3	1
112	0	256	150	3	0
113	0	335	143	3	0
114	2	218	134	1	0
115	0	307	146	3	0
116	0	254	147	3	0
117	0	177	120	3	0
118	1	204	202	2	1
119	1	295	162	2	1
120	1	309	156	3	1
121	0	169	144	1	0
122	0	282	142	3	0
123	2	417	157	2	1
124	0	260	140	3	0
125	3	240	171	2	1
126	0	302	151	2	1
127	2	197	152	2	1
128	2	223	169	2	1
129	0	192	148	1	1
130	3	240	171	2	1
131	2	256	149	2	1
132	1	203	132	1	1
133	2	196	163	2	1
134	1	309	156	3	1
135	0	225	146	1	0
136	1	342	166	2	1
137	0	325	154	2	1
138	2	257	156	2	1
139	1	235	153	2	1
140	0	274	88	3	0
141	0	197	136	2	0
142	3	234	145	2	0
143	3	182	174	2	1
144	0	204	143	2	1
145	0	167	114	3	0
146	2	295	157	2	1
147	0	172	158	3	0
148	3	298	178	3	1
149	2	321	182	2	1
150	0	318	140	1	0
151	1	283	195	3	0
152	0	300	171	3	0
153	2	258	147	3	1
154	0	330	132	3	0
155	1	309	156	3	1
156	3	199	178	3	1
157	2	258	147	3	1
158	2	564	160	3	1
159	1	157	182	2	1
160	0	304	162	2	0
161	2	222	143	2	1
162	0	304	162	2	0

163	0	274	166	3	0
164	0	249	144	2	0
165	0	326	140	3	0
166	0	184	105	1	0
167	0	354	163	2	1
168	2	315	162	2	1
169	1	160	138	2	1
170	0	247	171	2	1
171	0	283	103	3	0
172	1	240	169	2	1
173	2	199	179	2	1
174	0	239	126	3	0
175	0	288	133	3	0
176	0	283	103	3	0
177	0	246	96	2	0
178	0	197	177	2	0
179	0	409	150	3	0
180	0	187	144	3	0
181	3	211	144	2	1
182	0	293	170	3	0
183	2	180	150	2	1
184	1	308	170	2	1
185	0	289	124	3	0
186	0	167	114	3	0
187	0	249	144	2	0
188	0	254	147	3	0
189	2	309	131	3	0
190	2	250	179	2	1
191	1	221	163	3	1
192	2	564	160	3	1
193	3	234	131	2	1
194	0	286	108	2	0
195	2	212	157	2	1
196	0	234	156	3	0
197	0	260	185	2	1
198	2	318	160	2	1
199	0	200	126	3	0
200	0	209	163	2	1
201	3	182	174	2	1
202	3	298	178	3	1
203	3	227	155	3	1
204	2	278	152	2	1
205	3	244	178	2	1
206	2	218	134	1	0
207	2	250	179	2	1
208	2	175	173	2	1
209	1	281	103	3	0
210	2	240	194	3	1
211	0	299	125	2	0
212	0	243	128	3	0
213	2	315	162	2	1
214	1	308	170	2	1
215	1	266	171	2	1
216	0	254	127	3	0
217	1	157	182	2	1
218	0	311	120	3	0

219	0	286	116	2	0
220	1	236	174	2	0
221	0	254	147	3	0
222	3	211	144	2	1
223	2	199	179	2	1
224	0	261	186	2	1
225	2	232	165	3	1
226	2	149	126	2	0
227	2	242	149	2	1
228	1	205	184	2	1
229	0	228	165	3	0
230	0	300	171	3	0
231	1	236	178	2	1
232	0	258	141	3	0
233	1	306	163	2	1
234	0	269	163	2	1
235	3	227	155	3	1
236	2	149	126	2	0
237	1	261	141	3	0
238	0	230	160	3	0
239	0	244	154	2	0
240	1	288	159	2	1
241	2	212	150	3	0
242	2	175	173	2	1
243	2	185	155	2	0
244	2	245	166	2	1
245	1	219	188	2	1
246	1	283	195	3	0
247	0	311	120	3	0
248	2	220	152	2	1
249	2	180	150	2	1
250	0	275	118	2	0
251	1	160	138	2	1
252	0	353	132	3	0
253	0	289	124	3	0
254	0	198	130	3	0
255	0	394	157	2	1
256	0	183	182	2	1
257	0	325	154	2	1
258	3	231	182	3	0
259	0	302	151	2	1
260	2	226	169	2	1
261	2	232	165	3	1
262	0	222	186	2	1
263	1	342	166	2	1
264	0	225	146	1	0
265	0	208	148	2	1
266	0	282	95	3	0
267	0	237	71	2	0
268	2	224	173	3	0
269	2	265	130	2	1
270	0	211	161	3	1
271	1	263	173	3	1
272	2	220	152	2	1
273	0	206	108	2	0
274	0	228	138	1	1

275	0	283	103	3	0
276	0	207	168	3	1
277	1	219	188	2	1
278	0	289	145	3	0
279	1	198	168	2	1
280	1	234	175	2	1
281	1	192	174	2	1
282	1	204	172	2	1
283	3	211	144	2	1
284	2	224	173	3	0
285	2	265	130	2	1
286	2	313	133	3	1
287	1	302	162	2	1
288	2	340	172	2	1
289	0	223	181	3	0
290	2	223	169	2	1
291	0	259	130	3	0
292	2	243	137	2	1
293	2	212	157	2	1
294	0	288	133	3	0
295	0	299	125	2	0
296	0	237	71	2	0
297	0	270	111	3	0
298	1	192	174	2	1
299	1	325	172	2	1
300	1	204	172	2	1
301	2	227	154	3	1
302	1	342	166	2	1
303	0	282	142	3	0
304	2	196	169	2	1
305	0	267	99	3	0
306	2	242	149	2	1
307	1	220	170	2	1
308	2	218	134	1	0
309	1	294	153	2	1
310	0	260	140	3	0
311	0	256	150	3	0
312	2	269	112	3	0
313	1	269	121	2	1
314	3	199	178	3	1
315	3	244	178	2	1
316	2	313	133	3	1
317	2	252	172	2	1
318	0	177	162	3	0
319	2	216	115	0	1
320	0	264	143	2	1
321	2	275	139	2	1
322	0	309	147	3	0
323	1	246	120	1	0
324	1	245	180	2	1
325	1	294	153	2	1
326	1	283	195	3	0
327	0	276	112	1	0
328	0	322	109	2	0
329	2	216	115	0	1
330	2	215	170	2	1

331	0	269	169	2	0
332	2	250	187	2	1
333	2	214	158	2	1
334	0	206	132	3	0
335	0	270	111	3	0
336	2	126	173	3	1
337	2	273	152	2	1
338	2	256	142	1	0
339	0	253	144	3	0
340	2	175	173	2	1
341	2	226	169	2	1
342	2	269	148	2	1
343	2	199	162	3	1
344	1	157	182	2	1
345	1	246	120	1	0
346	0	243	128	3	0
347	2	267	167	2	1
348	0	247	143	3	0
349	2	263	97	3	0
350	0	302	151	2	1
351	0	200	126	3	0
352	0	335	143	3	0
353	0	201	126	1	1
354	1	261	141	3	0
355	0	243	152	2	1
356	0	176	90	1	0
357	0	286	108	2	0
358	3	204	162	2	0
359	2	216	115	0	1
360	0	222	186	2	1
361	2	231	146	3	1
362	2	213	165	2	1
363	2	246	173	2	1
364	1	236	174	2	0
365	2	246	173	2	1
366	2	230	165	3	0
367	1	229	168	3	0
368	2	240	154	3	1
369	2	175	123	2	1
370	0	341	136	3	0
371	0	353	132	3	0
372	2	214	158	2	1
373	1	284	160	2	0
374	2	177	160	2	1
375	0	228	138	1	1
376	1	221	164	2	1
377	0	303	122	2	1
378	0	237	71	2	0
379	3	186	190	1	1
380	0	218	105	3	0
381	2	224	173	3	0
382	0	239	142	3	0
383	0	270	111	3	0
384	0	282	156	3	0
385	2	175	123	2	1
386	2	209	173	2	1

387	0	304	162	2	0
388	0	246	96	2	0
389	3	233	150	1	1
390	1	319	152	2	0
391	3	264	132	3	0
392	2	175	123	2	1
393	0	164	145	3	0
394	0	330	132	3	0
395	2	278	152	2	1
396	2	274	150	3	0
397	0	167	114	3	0
398	0	228	138	1	1
399	3	233	150	1	1
400	2	188	139	3	0
401	0	149	125	2	1
402	1	245	143	2	1
403	1	204	172	2	1
404	0	207	138	3	0
405	2	256	142	1	0
406	2	211	165	2	1
407	0	234	156	3	0
408	0	243	152	2	1
409	2	231	147	2	0
410	1	198	168	2	1
411	0	249	144	2	0
412	0	276	112	1	0
413	0	322	109	2	0
414	3	288	159	3	0
415	1	204	172	2	1
416	2	273	152	2	1
417	2	223	169	2	1
418	0	209	163	2	1
419	2	360	151	2	1
420	0	303	159	2	1
421	0	265	122	2	1
422	0	354	163	2	1
423	1	319	152	2	0
424	0	309	147	3	0
425	0	305	142	3	0
426	2	201	163	2	1
427	2	168	174	2	1
428	0	247	143	3	0
429	2	243	152	2	0
430	2	212	150	3	0
431	0	225	114	3	0
432	2	318	160	2	1
433	2	250	187	2	1
434	2	268	172	2	1
435	0	354	163	2	1
436	0	249	143	2	0
437	0	229	129	3	0
438	2	253	179	2	1
439	1	284	160	2	0
440	0	244	154	2	0
441	0	293	170	3	0
442	0	274	88	3	0

443	2	168	174	2	1
444	2	253	179	2	1
445	1	205	184	2	1
446	2	246	173	2	1
447	0	289	145	3	0
448	2	295	157	2	1
449	0	230	160	2	0
450	0	407	154	3	0
451	1	271	162	2	1
452	0	228	165	3	0
453	1	271	162	2	1
454	0	225	114	3	0
455	3	234	131	2	1
456	2	243	152	2	0
457	2	220	152	2	1
458	0	247	171	2	1
459	0	261	186	2	1
460	2	254	146	3	0
461	2	255	175	2	1
462	3	186	190	1	1
463	0	211	161	3	1
464	2	564	160	3	1
465	2	175	173	2	1
466	1	219	188	2	1
467	0	275	118	2	0
468	2	243	137	2	1
469	0	286	108	2	0
470	3	240	171	2	1
471	2	313	133	3	1
472	0	197	131	2	1
473	2	214	168	2	1
474	1	229	168	3	0
475	2	126	173	3	1
476	0	289	124	3	0
477	2	229	150	3	0
478	2	321	182	2	1
479	0	216	131	3	0
480	0	305	142	3	0
481	0	407	154	3	0
482	0	298	122	3	0
483	1	192	174	2	1
484	0	248	158	1	0
485	1	281	103	3	0
486	0	172	158	3	0
487	0	254	127	3	0
488	1	288	159	2	1
489	2	243	137	2	1
490	0	303	159	2	1
491	2	168	174	2	1
492	2	309	131	3	0
493	0	353	132	3	0
494	2	245	166	2	1
495	0	234	161	3	1
496	2	274	150	3	0
497	1	232	164	2	0
498	0	239	160	2	1

499	2	177	160	2	1
500	0	149	125	2	1
501	0	183	182	2	1
502	2	177	160	2	1
503	1	234	175	2	1
504	2	243	152	2	0
505	2	242	149	2	1
506	0	260	140	3	0
507	1	204	172	2	1
508	0	288	133	3	0
509	0	327	117	2	0
510	1	288	159	2	1
511	0	177	120	3	0
512	0	290	153	2	0
513	0	206	108	2	0
514	1	220	170	2	1
515	2	188	139	3	0
516	0	206	132	3	0
517	1	198	168	2	1
518	2	188	139	3	0
519	0	203	161	3	0
520	0	177	162	3	0
521	1	220	144	3	1
522	2	277	172	2	1
523	0	203	161	3	0
524	2	230	165	3	0
525	2	256	149	2	1
526	0	164	145	3	0
527	0	209	163	2	1
528	3	270	145	3	1
529	3	234	131	2	1
530	0	258	157	3	0
531	2	269	148	2	1
532	0	197	136	2	0
533	0	234	160	2	1
534	2	267	167	2	1
535	2	197	116	2	1
536	2	219	158	2	1
537	1	325	172	2	1
538	0	249	144	3	0
539	3	227	155	3	1
540	0	259	130	3	0
541	2	235	180	2	1
542	0	394	157	2	1
543	3	204	162	2	0
544	2	273	152	2	1
545	1	229	168	3	0
546	0	322	109	2	0
547	0	223	142	2	1
548	2	295	157	2	1
549	2	277	151	3	1
550	2	254	146	3	0
551	0	286	116	2	0
552	0	341	136	3	0
553	2	197	152	2	1
554	0	234	156	3	0

555	0	254	163	3	0
556	0	177	162	3	0
557	0	222	186	2	1
558	2	199	179	2	1
559	0	237	71	2	0
560	0	197	131	2	1
561	2	269	148	2	1
562	2	209	173	2	1
563	0	290	153	2	0
564	0	184	105	1	0
565	0	234	160	2	1
566	0	254	159	2	1
567	2	214	168	2	1
568	2	201	163	2	1
569	2	240	194	3	1
570	2	304	170	2	1
571	0	282	142	3	0
572	3	182	174	2	1
573	0	290	153	2	0
574	0	258	141	3	0
575	0	247	171	2	1
576	3	298	178	3	1
577	0	322	109	2	0
578	0	394	157	2	1
579	0	218	105	3	0
580	1	197	156	3	1
581	2	235	180	2	1
582	1	262	155	2	1
583	0	177	120	3	0
584	0	353	132	3	0
585	3	199	178	3	1
586	2	309	131	3	0
587	0	176	90	1	0
588	0	307	146	3	0
589	0	286	116	2	0
590	1	269	121	2	1
591	0	269	169	2	0
592	2	269	112	3	0
593	0	269	169	2	0
594	0	212	132	1	0
595	0	203	161	3	0
596	1	221	164	2	1
597	2	175	173	2	1
598	1	284	160	2	0
599	1	195	179	2	1
600	2	263	97	3	0
601	0	311	120	3	0
602	2	340	172	2	1
603	1	195	179	2	1
604	2	253	179	2	1
605	2	265	130	2	1
606	0	212	132	2	0
607	0	315	125	1	0
608	0	212	132	1	0
609	0	327	117	2	0
610	0	341	136	3	0

611	0	205	130	3	0
612	0	225	146	1	0
613	0	217	111	3	0
614	0	305	142	3	0
615	2	219	158	2	1
616	0	303	181	2	1
617	1	306	163	2	1
618	1	266	171	2	1
619	0	248	158	1	0
620	0	274	88	3	0
621	0	256	150	3	0
622	0	294	106	2	0
623	3	234	145	2	0
624	3	270	145	3	1
625	2	254	146	3	0
626	2	224	173	3	0
627	3	231	182	3	0
628	3	239	151	2	1
629	3	282	174	2	0
630	3	264	132	3	0
631	1	266	171	2	1
632	1	234	175	2	1
633	0	166	125	2	0
634	0	212	168	3	0
635	0	264	143	2	1
636	0	249	143	2	0
637	2	340	172	2	1
638	3	282	174	2	0
639	0	197	131	2	1
640	0	243	152	2	1
641	0	409	150	3	0
642	0	263	105	3	1
643	0	177	140	3	1
644	2	226	169	2	1
645	0	243	128	3	0
646	2	243	152	2	0
647	0	303	122	2	1
648	0	149	125	2	1
649	1	234	175	2	1
650	0	267	99	3	0
651	1	157	182	2	1
652	3	226	114	2	1
653	0	283	103	3	0
654	1	306	163	2	1
655	1	235	153	2	1
656	1	236	174	2	0
657	2	220	152	2	1
658	2	309	131	3	0
659	0	271	182	2	1
660	0	166	125	2	0
661	0	318	140	1	0
662	0	204	143	2	1
663	0	248	122	2	1
664	3	226	114	2	1
665	2	417	157	2	1
666	1	192	174	2	1

667	1	261	141	3	0
668	1	204	202	2	1
669	1	246	120	1	0
670	0	330	169	2	0
671	0	212	168	3	0
672	2	277	151	3	1
673	2	258	147	3	1
674	0	330	132	3	0
675	0	234	156	3	0
676	0	253	144	3	0
677	0	254	147	3	0
678	2	268	172	2	1
679	2	274	150	3	0
680	1	295	162	2	1
681	0	326	140	3	0
682	0	176	90	1	0
683	0	177	120	3	0
684	2	185	155	2	0
685	0	407	154	3	0
686	0	204	156	0	0
687	0	300	171	3	0
688	0	288	133	3	0
689	2	304	170	2	1
690	2	240	154	3	1
691	1	250	161	2	1
692	0	203	155	3	0
693	1	195	179	2	1
694	0	219	140	3	0
695	0	282	156	3	0
696	2	219	158	2	1
697	2	212	150	3	0
698	0	212	132	2	0
699	0	282	156	3	0
700	2	214	168	2	1
701	0	198	130	3	0
702	1	302	162	2	1
703	0	201	126	1	1
704	2	227	154	3	1
705	0	216	131	3	0
706	2	229	150	3	0
707	1	294	153	2	1
708	2	178	96	2	1
709	3	264	132	3	0
710	1	221	163	3	1
711	0	198	130	3	0
712	1	160	138	2	1
713	3	226	114	2	1
714	3	213	125	2	1
715	1	245	143	2	1
716	0	205	130	3	0
717	2	256	142	1	0
718	1	250	161	2	1
719	0	233	147	3	1
720	2	335	158	2	0
721	0	260	185	2	1
722	2	277	172	2	1

723	2	211	115	2	1
724	1	269	121	2	1
725	0	258	157	3	0
726	0	274	166	3	0
727	1	221	163	3	1
728	0	311	120	3	0
729	1	250	161	2	1
730	1	220	170	2	1
731	0	230	160	2	0
732	2	227	154	3	1
733	2	141	175	2	1
734	0	204	156	0	0
735	2	196	163	2	1
736	0	239	142	3	0
737	0	229	129	3	0
738	0	300	171	3	0
739	0	255	161	3	0
740	2	235	180	2	1
741	2	268	172	2	1
742	0	330	132	3	0
743	1	220	144	3	1
744	2	318	160	2	1
745	2	222	143	2	1
746	2	335	158	2	0
747	0	230	160	3	0
748	2	226	169	2	1
749	1	220	144	3	1
750	1	262	155	2	1
751	2	360	151	2	1
752	1	245	180	2	1
753	0	177	140	3	1
754	2	256	149	2	1
755	2	255	175	2	1
756	0	246	96	2	0
757	0	228	138	1	1
758	1	204	172	2	1
759	0	330	169	2	0
760	0	276	112	1	0
761	0	269	163	2	1
762	1	221	163	3	1
763	3	283	162	2	1
764	0	187	144	3	0
765	0	335	143	3	0
766	0	335	143	3	0
767	0	193	141	3	0
768	1	197	156	3	1
769	2	265	130	2	1
770	1	203	132	1	1
771	0	236	152	2	1
772	0	244	154	2	0
773	0	225	114	3	0
774	2	275	139	2	1
775	2	222	143	2	1
776	0	307	146	3	0
777	0	282	95	3	0
778	3	204	162	2	0

779	1	210	192	2	1
780	0	169	144	1	0
781	0	218	105	3	0
782	0	303	122	2	1
783	1	240	169	2	1
784	2	232	165	3	1
785	0	289	145	3	0
786	0	254	163	3	0
787	0	298	122	3	0
788	0	294	106	2	0
789	1	281	103	3	0
790	0	239	126	3	0
791	0	239	126	3	0
792	0	193	141	3	0
793	2	178	96	2	1
794	3	234	145	2	0
795	1	208	140	2	1
796	1	203	132	1	1
797	0	225	114	3	0
798	3	288	159	3	0
799	0	303	181	2	1
800	0	229	129	3	0
801	3	233	150	1	1
802	0	197	136	2	0
803	0	230	160	2	0
804	0	197	131	2	1
805	0	226	111	3	1
806	0	276	112	1	0
807	2	233	179	2	1
808	2	227	154	3	1
809	2	214	158	2	1
810	0	167	114	3	0
811	1	261	141	3	0
812	0	268	160	2	0
813	0	203	155	3	0
814	1	208	140	2	1
815	2	240	154	3	1
816	1	245	143	2	1
817	0	260	185	2	1
818	3	244	178	2	1
819	0	225	146	1	0
820	0	207	138	3	0
821	0	268	160	2	0
822	0	253	144	3	0
823	0	239	160	2	1
824	0	166	125	2	0
825	2	252	172	2	1
826	2	180	150	2	1
827	2	229	150	3	0
828	2	233	179	2	1
829	0	266	109	3	0
830	2	222	143	2	1
831	1	220	144	3	1
832	2	277	151	3	1
833	0	217	111	3	0
834	0	315	125	1	0

835	2	149	126	2	0
836	0	234	160	2	1
837	2	199	162	3	1
838	3	213	125	2	1
839	0	261	186	2	1
840	0	174	125	3	0
841	0	183	182	2	1
842	2	230	165	3	0
843	3	273	125	2	0
844	0	293	170	3	0
845	0	184	105	1	0
846	0	183	182	2	1
847	0	166	125	2	0
848	3	283	162	2	1
849	0	255	161	3	0
850	1	284	160	2	0
851	2	250	187	2	1
852	0	255	161	3	0
853	0	229	129	3	0
854	3	282	174	2	0
855	1	197	156	3	1
856	2	211	115	2	1
857	0	303	181	2	1
858	2	211	115	2	1
859	0	299	173	3	0
860	0	230	160	2	0
861	2	335	158	2	0
862	3	288	159	3	0
863	0	212	168	3	0
864	3	273	125	2	0
865	3	240	171	2	1
866	2	250	179	2	1
867	1	235	153	2	1
868	1	240	169	2	1
869	1	236	178	2	1
870	2	275	139	2	1
871	2	321	182	2	1
872	3	227	155	3	1
873	0	192	148	1	1
874	3	273	125	2	0
875	0	206	132	3	0
876	0	207	138	3	0
877	2	213	165	2	1
878	0	188	113	3	0
879	0	271	182	2	1
880	0	207	168	3	1
881	1	232	164	2	0
882	0	131	115	3	0
883	0	274	166	3	0
884	0	174	125	3	0
885	0	289	124	3	0
886	0	260	140	3	0
887	0	201	126	1	1
888	0	258	157	3	0
889	0	407	154	3	0
890	0	205	130	3	0

891	0	325	154	2	1
892	0	239	126	3	0
893	0	204	156	0	0
894	0	299	173	3	0
895	2	263	97	3	0
896	3	270	145	3	1
897	1	201	158	2	1
898	0	265	122	2	1
899	0	234	161	3	1
900	3	234	145	2	0
901	0	265	122	2	1
902	0	268	160	2	0
903	2	218	134	1	0
904	1	262	155	2	1
905	0	246	96	2	0
906	0	226	178	2	1
907	1	244	162	2	1
908	0	267	99	3	0
909	0	200	126	3	0
910	2	233	163	3	0
911	1	319	152	2	0
912	0	198	130	3	0
913	0	208	148	2	1
914	0	212	132	2	0
915	0	249	144	3	0
916	0	254	127	3	0
917	2	253	179	2	1
918	3	204	162	2	0
919	3	231	182	3	0
920	0	219	140	3	0
921	0	218	105	3	0
922	1	263	173	3	1
923	0	239	160	2	1
924	0	330	169	2	0
925	0	131	115	3	0
926	0	206	108	2	0
927	2	240	194	3	1
928	0	266	109	3	0
929	0	206	132	3	0
930	0	254	127	3	0
931	0	223	181	3	0
932	2	308	142	2	1
933	3	231	182	3	0
934	2	180	150	2	1
935	1	240	169	2	1
936	2	315	162	2	1
937	2	335	158	2	0
938	0	226	111	3	1
939	1	271	162	2	1
940	0	241	123	3	0
941	2	196	169	2	1
942	3	239	151	2	1
943	0	177	140	3	1
944	0	228	165	3	0
945	3	193	162	3	1
946	2	277	172	2	1

947	2	201	163	2	1
948	0	174	125	3	0
949	0	207	168	3	1
950	0	286	108	2	0
951	2	263	97	3	0
952	2	304	170	2	1
953	0	236	152	2	1
954	0	264	143	2	1
955	2	231	146	3	1
956	0	269	163	2	1
957	2	233	163	3	0
958	2	417	157	2	1
959	3	239	151	2	1
960	2	196	169	2	1
961	0	248	122	2	1
962	0	233	147	3	1
963	0	241	123	3	0
964	2	141	175	2	1
965	2	197	116	2	1
966	0	259	130	3	0
967	2	178	96	2	1
968	0	203	155	3	0
969	1	325	172	2	1
970	2	175	173	2	1
971	2	199	162	3	1
972	3	186	190	1	1
973	2	245	166	2	1
974	0	211	161	3	1
975	0	219	140	3	0
976	0	269	169	2	0
977	1	205	184	2	1
978	0	197	177	2	0
979	0	309	147	3	0
980	0	192	148	1	1
981	0	219	140	3	0
982	0	223	142	2	1
983	0	263	105	3	1
984	0	234	161	3	1
985	2	231	146	3	1
986	0	327	117	2	0
987	1	232	164	2	0
988	0	293	170	3	0
989	1	302	162	2	1
990	1	236	178	2	1
991	0	230	160	3	0
992	0	254	159	2	1
993	0	247	143	3	0
994	0	239	142	3	0
995	1	263	173	3	1
996	0	409	150	3	0
997	0	188	113	3	0
998	0	315	125	1	0
999	0	254	163	3	0
1000	0	212	132	1	0
1001	0	226	178	2	1
1002	0	212	132	2	0

```
1003   0   233      147     3       1
1004   2   308      142     2       1
1005   0   205      130     3       0
1006   2   211      165     2       1
1007   3   193      162     3       1
1008   1   295      162     2       1
1009   0   223      181     3       0
1010   0   299      173     3       0
1011   1   308      170     2       1
1012   1   229      168     3       0
1013   0   318      140     1       0
1014   2   141      175     2       1
1015   0   216      131     3       0
1016   3   282      174     2       0
1017   0   282       95     3       0
1018   0   172      158     3       0
1019   0   204      143     2       1
1020   1   221      164     2       1
1021   0   258      141     3       0
1022   0   275      118     2       0
1023   0   254      159     2       1
1024   0   188      113     3       0
```

Step 3: Understand and Preprocess Sample Data

Step 3.1: Understand Sample Data

- **Objective:** Check dataset dimensions, data types, missing values (`null` / `NaN`), and distribution of target labels.

```
"""
Purpose of this section:
-------------------------
This part of the code helps us understand the dataset by:
1. Printing the names of all attributes (columns) in the DataFrame.
2. Showing the total number of instances (rows) available in the dataset.
"""

# Print a header for clarity
print("\n\nAttributes in Heart Disease Data:")
print("================================\n")

# Display all column names (attributes) in the dataset
print(heart_data.columns)

# Print a header for instance count (using 'target' column to count rows)
print("\n\nNumber of Instances in Heart Disease Data:", heart_data["target"].count())
print("==========================================\n")
```

```
Attributes in Heart Disease Data:
================================

Index(['cp', 'chol', 'thalach', 'thal', 'target'], dtype='str')


Number of Instances in Heart Disease Data: 1025
==============================================
```

Step 3.2: Process Sample Data

- **Status: Already Completed in `heart.csv`**
- **Reasoning:** Unlike the raw Titanic dataset (which required handling missing values in `Age` or `Cabin` and dropping irrelevant text columns), `heart.csv` is a clean, pre-cleaned benchmark medical dataset. All records contain complete clinical values without missing entries.

Step 4: Feature Extraction

- **Status: Already Completed for This Project**
- **Reasoning:** In text or complex tabular datasets (like Titanic's `Name` or `Ticket` columns), custom feature extraction is needed to derive titles or family sizes. For `heart.csv`, feature selection is driven by clinical domain knowledge: the key medical attributes (`cp`, `chol`, `thalach`, `thal`) are already extracted into structured, independent numerical variables.

Step 5: Label Encoding the Sample Data

- **Status: Not Needed / Skipped for Heart Disease Dataset**
- **Reasoning & Adaptation:**
  - **Titanic Workflow:** Used `LabelEncoder` to translate text categorical columns (`Gender`: `'male'` / `'female'`, `Embarked`: `'Cherbourg'` / `'Southampton'`) into integers $(0, 1, 2)$.
  - **Heart Disease Workflow:** All features in `heart.csv` are already purely numeric (`cp` $= 0 - 3$, `chol` $= \text{mg/dl}$, `thalach` $= \text{bpm}$, `thal` $= 0 - 3$).
  - **Alternative Requirement (Feature Scaling):** Because Support Vector Machines calculate geometric distances between data points, features with large scales (like cholesterol ≈ 250) would dominate small features (like chest pain ≈ 2). Instead of Label Encoding, we apply **`StandardScaler`** during Step 6 to standardize features to a zero mean ($\mu = 0$) and unit variance ($\sigma = 1$).

Step 6: Execute the Training Phase

Step 6.1: Splitting Sample Data into Training and Testing Data

- Splits the dataset into an **80% Training Set** (for model learning) and a **20% Testing Set** (for evaluation) using `train_test_split` with `random_state=42` and `shuffle=True`.

```
"""
Purpose of this section:
------------------------
```

```
This part of the code splits the heart disease dataset into Training Data and Testing Data.
- Training Data is used to teach the machine learning model.
- Testing Data is used to check how well the model performs on unseen data.
"""

# Split dataset into training (80%) and testing (20%)
# shuffle=True (default) ensures data is randomized before splitting
training_data, testing_data = train_test_split(
    heart_data,
    test_size=0.2,
    random_state=42,
    shuffle=True
)

# Save the training and testing datasets to CSV files
training_data.to_csv(r'training-data.csv', index=False, header=True)
testing_data.to_csv(r'testing-data.csv', index=False, header=True)

print("Training Data shape:", training_data.shape)
print("Testing Data shape:", testing_data.shape)
```

```
Training Data shape: (820, 5)
Testing Data shape: (205, 5)
```

Step 6.2: Splitting Input Vectors and Output Labels of Training Data

- Isolates input feature columns (X_{train}: `cp`, `chol`, `thalach`, `thal`) from the target column (y_{train}: `target`).

```
"""
Purpose of this section:
------------------------
This part of the code separates the features (input vectors) and
the labels (output values) from the training dataset.
"""

# Extract input vectors (features) from training data
input_vector_train = training_data.iloc[:, :-1]

# Extract output labels (target variable) from training data
output_label_train = training_data.iloc[:, -1]

print("Input vectors (train) shape:", input_vector_train.shape)
print("Output labels (train) shape:", output_label_train.shape)
```

```
Input vectors (train) shape: (820, 4)
Output labels (train) shape: (820,)
```

Step 6.3: Train the Support Vector Classifier (SVC)

- Fits a `StandardScaler` on X_{train} and transforms input vectors into scaled Z-scores ($X_{\text{train_scaled}}$).
- Initializes and fits the `SVC(kernel='rbf')` model on scaled training vectors and target labels.

```
"""
Purpose of this section:
------------------------
This part of the code trains a Support Vector Classifier (SVC) on the heart disease training
```

```
data.
Features are scaled using StandardScaler so large feature values (e.g., cholesterol)
do not distort the model's decision boundary.`
"""

from sklearn.preprocessing import StandardScaler

print("\n\nTraining the Support Vector Classifier on Training Data")
print("=========================================================\n")

# Scale feature inputs so all attributes have mean = 0 and variance = 1
scaler = StandardScaler()
input_vector_train_scaled = scaler.fit_transform(input_vector_train)

# Initialize the Support Vector Classifier with standard parameters
svc_model = svm.SVC(gamma='auto', random_state=42)

# Train the model using the scaled features and flattened labels
svc_model.fit(input_vector_train_scaled, np.ravel(output_label_train))

# Print trained model details
print(svc_model)
```

```
Training the Support Vector Classifier on Training Data
=========================================================

SVC(gamma='auto', random_state=42)
```

Step 6.4: Save the Trained Model & Scaler

- Serializes both trained artifacts (`svc_trained_model.pkl` and `scaler.pkl`) to disk using `pickle.dump()` .

```
"""
Purpose of this section:
------------------------
This part of the code saves both the trained Support Vector Classifier (SVC)
model and the fitted StandardScaler to your computer so they can be reused
later without retraining.
"""

# Save the trained SVC model to disk
pickle.dump(svc_model, open('svc_trained_model.pkl', 'wb'))
print("Saved: svc_trained_model.pkl")

# Save the fitted scaler to disk (crucial for scaling future patient inputs)
pickle.dump(scaler, open('scaler.pkl', 'wb'))
print("Saved: scaler.pkl")
```

```
Saved: svc_trained_model.pkl
Saved: scaler.pkl
```

Step 7: Execute the Testing Phase

Step 7.1: Splitting Input Vectors and Output Labels of Testing Data

- Isolates testing input features (X_{test}) and true target outcomes (y_{test}).

```
"""
Purpose of this section:
------------------------
This part of the code separates the features (inputs) and the labels (outputs)
from the testing dataset.
"""

input_vector_test = testing_data.iloc[:, :-1]
output_label_test = testing_data.iloc[:, -1]

print("Input vectors (test) shape:", input_vector_test.shape)
print("Output labels (test) shape:", output_label_test.shape)
```

```
Input vectors (test) shape: (205, 4)
Output labels (test) shape: (205,)
```

Step 7.2: Load the Saved Model

- Reloads `svc_trained_model.pkl` and `scaler.pkl` from disk to verify artifact integrity

```
"""
Purpose of this section:
------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model and the StandardScaler from disk into memory.
"""

# Load trained SVC model from disk
model = pickle.load(open('svc_trained_model.pkl', 'rb'))
print("Model loaded from svc_trained_model.pkl")

# Load fitted StandardScaler from disk
scaler = pickle.load(open('scaler.pkl', 'rb'))
print("Scaler loaded from scaler.pkl")
```

```
Model loaded from svc_trained_model.pkl
Scaler loaded from scaler.pkl
```

Step 7.3: Evaluate the Machine Learning Model

Step 7.3.1: Make Predictions with the Trained Model on Testing Data

- Scales test features using `scaler.transform(X_test)` and generates predictions using `model.predict()`.

```
"""
Purpose of this section:
------------------------
This part of the code evaluates the trained Support Vector Classifier (SVC)
on the heart disease testing data using scaled features.
"""

# Scale test features (converts DataFrame to scaled NumPy array)
input_vector_test_scaled = scaler.transform(input_vector_test)
```

```
# Predict targets on SCALED test features
model_predictions = model.predict(input_vector_test_scaled)

# Suppress pandas chained assignment warnings
pd.options.mode.chained_assignment = None

# Append predictions to 'testing_data' (not testing_data_encoded)
testing_data["Predictions"] = model_predictions

# Save full results and predictions to a CSV file
testing_data.to_csv(r'model-predictions.csv', index=False, header=True)

# Print full prediction DataFrame
print("\n\nPredictions Returned by svc_trained_model:")
print("=========================================\n")
print(testing_data)
```

Predictions Returned by svc_trained_model:
==========

	cp	chol	thalach	thal	target	Predictions
527	0	209	163	2	1	1
359	2	216	115	0	1	0
447	0	289	145	3	0	0
31	1	244	162	2	1	1
621	0	256	150	3	0	0
590	1	269	121	2	1	1
905	0	246	96	2	0	0
737	0	229	129	3	0	0
76	2	255	175	2	1	1
948	0	174	125	3	0	0
432	2	318	160	2	1	1
70	0	326	140	3	0	0
680	1	295	162	2	1	1
101	2	126	173	3	1	1
845	0	184	105	1	0	0
656	1	236	174	2	0	1
310	0	260	140	3	0	0
344	1	157	182	2	1	1
837	2	199	162	3	1	1
174	0	239	126	3	0	0
96	0	303	159	2	1	1
822	0	253	144	3	0	0
598	1	284	160	2	0	1
218	0	311	120	3	0	0
420	0	303	159	2	1	1
208	2	175	173	2	1	1
774	2	275	139	2	1	1
290	2	223	169	2	1	1
889	0	407	154	3	0	0
670	0	330	169	2	0	1
903	2	218	134	1	0	0
1004	2	308	142	2	1	1
1012	1	229	168	3	0	0
329	2	216	115	0	1	0
1001	0	226	178	2	1	1
762	1	221	163	3	1	1
490	0	303	159	2	1	1
350	0	302	151	2	1	1
500	0	149	125	2	1	0
307	1	220	170	2	1	1
853	0	229	129	3	0	0
351	0	200	126	3	0	0
643	0	177	140	3	1	0
439	1	284	160	2	0	1
819	0	225	146	1	0	0
833	0	217	111	3	0	0
518	2	188	139	3	0	1
554	0	234	156	3	0	0
289	0	223	181	3	0	0
292	2	243	137	2	1	1
213	2	315	162	2	1	1

966	0	259	130	3	0	0
59	1	232	164	2	0	1
692	0	203	155	3	0	0
321	2	275	139	2	1	1
10	0	149	125	2	1	0
331	0	269	169	2	0	1
334	0	206	132	3	0	0
918	3	204	162	2	0	1
86	2	211	165	2	1	1
332	2	250	187	2	1	1
743	1	220	144	3	1	1
450	0	407	154	3	0	0
468	2	243	137	2	1	1
305	0	267	99	3	0	0
30	0	169	144	1	0	1
558	2	199	179	2	1	1
405	2	256	142	1	0	0
54	0	217	111	3	0	0
23	2	252	172	2	1	1
1018	0	172	158	3	0	0
481	0	407	154	3	0	0
3	0	203	161	3	0	0
231	1	236	178	2	1	1
355	0	243	152	2	1	1
673	2	258	147	3	1	0
756	0	246	96	2	0	0
843	3	273	125	2	0	1
667	1	261	141	3	0	0
215	1	266	171	2	1	1
634	0	212	168	3	0	0
318	0	177	162	3	0	0
986	0	327	117	2	0	0
352	0	335	143	3	0	0
715	1	245	143	2	1	1
394	0	330	132	3	0	0
198	2	318	160	2	1	1
594	0	212	132	1	0	0
863	0	212	168	3	0	0
924	0	330	169	2	0	1
39	2	229	150	3	0	1
790	0	239	126	3	0	0
314	3	199	178	3	1	1
761	0	269	163	2	1	1
580	1	197	156	3	1	1
66	2	197	152	2	1	1
717	2	256	142	1	0	0
67	0	315	125	1	0	0
294	0	288	133	3	0	0
921	0	218	105	3	0	0
464	2	564	160	3	1	1
107	1	281	103	3	0	0
629	3	282	174	2	0	1
576	3	298	178	3	1	0
254	0	198	130	3	0	0
723	2	211	115	2	1	1
929	0	206	132	3	0	0

539	3	227	155	3	1	1
915	0	249	144	3	0	0
525	2	256	149	2	1	1
543	3	204	162	2	0	1
754	2	256	149	2	1	1
100	2	213	165	2	1	1
88	0	268	160	2	0	1
63	2	212	157	2	1	1
848	3	283	162	2	1	1
893	0	204	156	0	0	0
139	1	235	153	2	1	1
603	1	195	179	2	1	1
620	0	274	88	3	0	0
542	0	394	157	2	1	1
494	2	245	166	2	1	1
589	0	286	116	2	0	0
109	0	206	108	2	0	0
962	0	233	147	3	1	0
247	0	311	120	3	0	0
922	1	263	173	3	1	0
697	2	212	150	3	0	1
486	0	172	158	3	0	0
136	1	342	166	2	1	1
932	2	308	142	2	1	1
688	0	288	133	3	0	0
758	1	204	172	2	1	1
158	2	564	160	3	1	1
493	0	353	132	3	0	0
501	0	183	182	2	1	1
682	0	176	90	1	0	0
973	2	245	166	2	1	1
256	0	183	182	2	1	1
734	0	204	156	0	0	0
857	0	303	181	2	1	1
959	3	239	151	2	1	1
617	1	306	163	2	1	1
983	0	263	105	3	1	0
901	0	265	122	2	1	0
210	2	240	194	3	1	0
168	2	315	162	2	1	1
939	1	271	162	2	1	1
440	0	244	154	2	0	1
1022	0	275	118	2	0	0
545	1	229	168	3	0	0
55	0	217	111	3	0	0
120	1	309	156	3	1	0
616	0	303	181	2	1	1
25	0	307	146	3	0	0
72	0	249	144	2	0	1
532	0	197	136	2	0	1
44	1	342	166	2	1	1
296	0	237	71	2	0	0
199	0	200	126	3	0	0
753	0	177	140	3	1	0
793	2	178	96	2	1	1
485	1	281	103	3	0	0

209	1	281	103	3	0	0
265	0	208	148	2	1	1
377	0	303	122	2	1	0
894	0	299	173	3	0	0
788	0	294	106	2	0	0
528	3	270	145	3	1	0
451	1	271	162	2	1	1
595	0	203	161	3	0	0
745	2	222	143	2	1	1
965	2	197	116	2	1	1
526	0	164	145	3	0	0
306	2	242	149	2	1	1
319	2	216	115	0	1	0
60	1	204	202	2	1	1
482	0	298	122	3	0	0
854	3	282	174	2	0	1
615	2	219	158	2	1	1
541	2	235	180	2	1	1
428	0	247	143	3	0	0
184	1	308	170	2	1	1
881	1	232	164	2	0	1
982	0	223	142	2	1	1
599	1	195	179	2	1	1
906	0	226	178	2	1	1
778	3	204	162	2	0	1
261	2	232	165	3	1	1
342	2	269	148	2	1	1
522	2	277	172	2	1	1
237	1	261	141	3	0	0
110	0	197	177	2	0	1
497	1	232	164	2	0	1
516	0	206	132	3	0	0
361	2	231	146	3	1	1
429	2	243	152	2	0	1
1013	0	318	140	1	0	0
277	1	219	188	2	1	1
260	2	226	169	2	1	1
832	2	277	151	3	1	0
796	1	203	132	1	1	0
644	2	226	169	2	1	1
404	0	207	138	3	0	0
842	2	230	165	3	0	1

Step 7.4: Calculate the Accuracy Score & Export Predictions

- Appends model predictions directly to the test DataFrame and exports the output to `model-predictions.csv`.

```
"""
Purpose of this section:
------------------------
Evaluates the SVM model on heart disease test data using:
- Classification Report: Precision, Recall, and F1-Score for each class.
- Confusion Matrix: Grid showing correct vs. incorrect diagnoses.
- Heatmap Visualization: Graphical representation of the confusion matrix.
"""
```

```python
# Generate classification report
print("\nClassification Report:")
print("======================\n")
print(classification_report(
    testing_data["target"],
    testing_data["Predictions"],
    target_names=["No Heart Disease (0)", "Heart Disease (1)"]
))

# Generate confusion matrix table
conf_matrix = confusion_matrix(testing_data["target"], testing_data["Predictions"])
conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual: No Heart Disease", "Actual: Heart Disease"],
    columns=["Predicted: No Heart Disease", "Predicted: Heart Disease"]
)

print("\nConfusion Matrix:")
print("=================\n")
print(conf_matrix_df)

# Plot confusion matrix heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(
    conf_matrix,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No Heart Disease', 'Heart Disease'],
    yticklabels=['No Heart Disease', 'Heart Disease']
)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - SVC on Heart Disease Data')
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================

                      precision    recall  f1-score   support

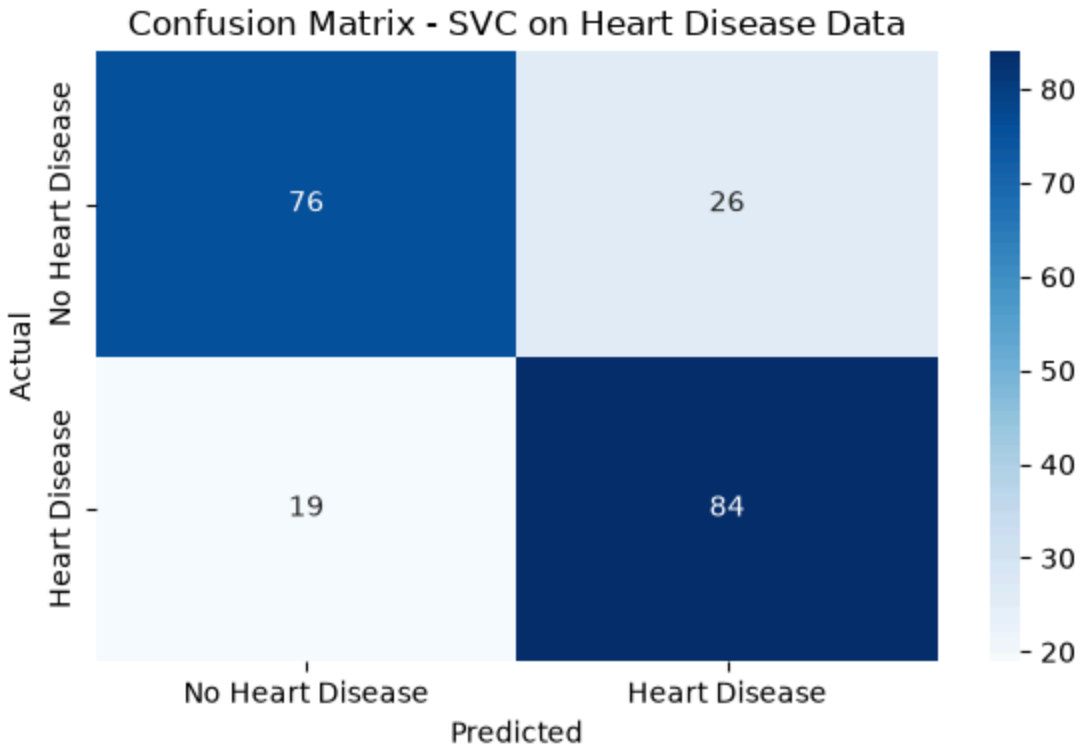
No Heart Disease (0)       0.80      0.75      0.77       102
   Heart Disease (1)       0.76      0.82      0.79       103

            accuracy                           0.78       205
           macro avg       0.78      0.78      0.78       205
        weighted avg       0.78      0.78      0.78       205


Confusion Matrix:
=================

                         Predicted: No Heart Disease  \
Actual: No Heart Disease                           76
Actual: Heart Disease                              19

                         Predicted: Heart Disease
Actual: No Heart Disease                        26
Actual: Heart Disease                           84
```



Step 7.5: Detailed Evaluation Metrics

- Generates a **Classification Report** (Precision, Recall, F1-score) and visualizes a **Confusion Matrix Heatmap** to assess diagnostic accuracy and false negative rates.

```
"""
Purpose of this section:
------------------------
This part of the code gathers clinical input data for a single patient
```

```
(Chest Pain, Cholesterol, Max Heart Rate, Thalassemia) to predict heart disease.

To enter custom values interactively during a live notebook session,
replace the four demo variables below with:
    cp_input = int(input("Enter Chest Pain Type (0-3): ").strip())
    chol_input = float(input("Enter Serum Cholesterol in mg/dl (e.g., 233): ").strip())
    thalach_input = float(input("Enter Maximum Heart Rate (e.g., 150): ").strip())
    thal_input = int(input("Enter Thalassemia Type (0-3): ").strip())
"""

# Demo patient medical features
cp_input = 2          # Chest Pain Type (0, 1, 2, or 3)
chol_input = 233.0    # Serum Cholesterol in mg/dl
thalach_input = 150.0 # Maximum Heart Rate achieved
thal_input = 2        # Thalassemia (0, 1, 2, or 3)

# Display summary of entered patient data
print(f"Patient Data -> Chest Pain (cp): {cp_input}, Cholesterol: {chol_input} mg/dl, Max Heart
Rate: {thalach_input} bpm, Thalassemia (thal): {thal_input}")
```

```
Patient Data -> Chest Pain (cp): 2, Cholesterol: 233.0 mg/dl, Max Heart Rate: 150.0 bpm, Thalasse
mia (thal): 2
```

Step 8: Execute the Application Phase

Step 8.1: Take Input from User

- Prompts or assigns patient clinical values: Chest Pain Type (`cp`), Serum Cholesterol (`chol`), Maximum Heart Rate (`thalach`), and Thalassemia (`thal`).

Step 8.2: Convert User Input into Feature Vector

- Formats raw user input into a single-row Pandas DataFrame matching the exact feature column schema of the training set.

```
"""
Purpose of this section:
------------------------
This part of the code takes the patient's clinical inputs and
converts them into a structured feature vector (DataFrame)
matching the feature columns of the heart disease dataset.
"""

# Create a single-row DataFrame using heart disease feature names
user_input = pd.DataFrame({
    'cp': [cp_input],
    'chol': [chol_input],
    'thalach': [thalach_input],
    'thal': [thal_input]
})

print("\n\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

   cp   chol  thalach  thal
0   2  233.0    150.0     2
```

Step 8.3: Label Encoding of Feature Vectors

- **Status: Not Needed (Replaced by Scaling)**
- **Reasoning:** Since inputs are already numeric, label encoding is skipped. Instead, the raw vector is transformed via `scaler.transform(user_input)` to match the standardized feature distribution expected by the SVM.

Step 8.4: Load the Saved Model

- Loads the production model (`svc_trained_model.pkl`) and scaler (`scaler.pkl`) into active memory.

```
"""
Purpose of this section:
------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model and the StandardScaler from disk into memory for the Application Phase.
"""

# Load the trained Support Vector Classifier model
model = pickle.load(open('svc_trained_model.pkl', 'rb'))
print("Model loaded from svc_trained_model.pkl")

# Load the fitted StandardScaler
scaler = pickle.load(open('scaler.pkl', 'rb'))
print("Scaler loaded from scaler.pkl")
```

```
Model loaded from svc_trained_model.pkl
Scaler loaded from scaler.pkl
```

Step 8.5: Model Prediction

Step 8.5.1: Apply Model on Scaled Features of Unseen Instances & Return Predictions to User

- Feeds the scaled single-patient vector into `model.predict()` , maps the numeric prediction (0 or 1) to clinical text (`"HEART DISEASE DETECTED"` vs. `"NO HEART DISEASE DETECTED"`), and prints a formatted output table.

```
"""
Purpose of this section:
------------------------
This part of the code scales the patient feature vector using the loaded
scaler, then uses the trained Support Vector Classifier (SVC) to predict
whether the patient has heart disease.
"""


# 1. Scale the patient features using the loaded scaler object
unseen_data_features_scaled = scaler.transform(user_input)
```

```
# 2. Predict using the SCALED feature vector
predicted_diagnosis = model.predict(unseen_data_features_scaled)

# 3. Map predicted numeric class (0 or 1) to clinical text output
if predicted_diagnosis[0] == 1:
    prediction = "HEART DISEASE DETECTED"
else:
    prediction = "NO HEART DISEASE DETECTED"

# 4. Create formatted ASCII table output
pretty_table = PrettyTable()
pretty_table.add_column("        ** Diagnosis Prediction **        ", [prediction])

print(pretty_table)
```

```
+-----------------------------------------+
|         ** Diagnosis Prediction **      |
+-----------------------------------------+
|           HEART DISEASE DETECTED        |
+-----------------------------------------+
```

Step 9: Execute the Feedback Phase

Step 9.1: Collect Feedback from Users and Domain Experts

- Compiles qualitative evaluation from cardiology experts and clinical testers reviewing specific model predictions against historical medical baselines.

```
"""
Purpose of this section:
------------------------
Step 9.1: Example feedback collected from a cardiology domain expert
and clinical testers reviewing this model's predictions on real-world-style patient cases.
"""

import pandas as pd

example_feedback = pd.DataFrame({
    "Reviewer": [
        "Cardiologist (Domain Expert)",
        "Medical Student Tester",
        "Clinical Analyst"
    ],
    "Case": [
        "Severe chest pain (cp=3), High cholesterol (300 mg/dl) -> predicted HEART DISEASE
DETECTED",
        "Asymptomatic (cp=0), High exercise heart rate (175 bpm) -> predicted NO HEART DISEASE",
        "Borderline cholesterol (240 mg/dl), Asymptomatic (cp=0) -> predicted HEART DISEASE
DETECTED (unexpected)"
    ],
    "Feedback": [
        "Matches clinical expectations: non-anginal/severe chest pain paired with high
cholesterol strongly correlates with coronary artery disease.",
        "Matches clinical baseline: high exercise tolerance and absence of chest pain typical of
healthy cardiac function.",
```

```
        "Borderline case - model may be overly sensitive to cholesterol thresholds when chest
pain is absent. Suggest collecting more sample cases to refine the decision boundary."
    ]
})

# Print clean formatted output without row index numbers
print(example_feedback.to_string(index=False))
```

```
                   Reviewer
Case
Feedback
Cardiologist (Domain Expert)              Severe chest pain (cp=3), High cholesterol (300 mg/dl)
-> predicted HEART DISEASE DETECTED                                    Matches clinical expectations:
non-anginal/severe chest pain paired with high cholesterol strongly correlates with coronary arte
ry disease.
      Medical Student Tester                    Asymptomatic (cp=0), High exercise heart rate (17
5 bpm) -> predicted NO HEART DISEASE                                                          Mat
ches clinical baseline: high exercise tolerance and absence of chest pain typical of healthy card
iac function.
           Clinical Analyst Borderline cholesterol (240 mg/dl), Asymptomatic (cp=0) -> predicted
HEART DISEASE DETECTED (unexpected) Borderline case - model may be overly sensitive to cholestero
l thresholds when chest pain is absent. Suggest collecting more sample cases to refine the decisi
on boundary.
```

Step 9.2: Potential Improvements

- Documents actionable model enhancements, such as expanding to all 13 clinical features, applying Stratified K-Fold Cross-Validation, and executing hyperparameter optimization (`GridSearchCV`).

```
"""`
Purpose of this section:
-------------------------
Step 9.2: Potential improvements identified for the Heart Disease SVM Model
based on clinical feedback and performance evaluation.
"""

improvements = [
    "Include all 13 clinical features from the UCI dataset (e.g., age, trestbps, fbs, oldpeak,
exang) rather than limiting to the top 4 attributes.",
    "Apply Stratified K-Fold Cross-Validation (e.g., k=5 or k=10) to ensure robust performance
metrics across varied patient subgroups.",
    "Perform Hyperparameter Tuning on the SVM classifier using GridSearchCV to optimize C, gamma,
and kernel selection (rbf vs. linear vs. poly).",
    "Compare performance against ensemble algorithms like Random Forest, XGBoost, and LightGBM to
assess non-linear decision boundaries.",
    "Incorporate probability calibration (SVC with probability=True) to output clinical
confidence scores rather than hard binary classifications."
]

print("\n\nPotential Model & Workflow Improvements:")
print("========================================\n")
for i, item in enumerate(improvements, 1):
    print(f"{i}. {item}")
```

```
Potential Model & Workflow Improvements:
========================================

1. Include all 13 clinical features from the UCI dataset (e.g., age, trestbps, fbs, oldpeak, exan
g) rather than limiting to the top 4 attributes.
2. Apply Stratified K-Fold Cross-Validation (e.g., k=5 or k=10) to ensure robust performance metr
ics across varied patient subgroups.
3. Perform Hyperparameter Tuning on the SVM classifier using GridSearchCV to optimize C, gamma, a
nd kernel selection (rbf vs. linear vs. poly).
4. Compare performance against ensemble algorithms like Random Forest, XGBoost, and LightGBM to a
ssess non-linear decision boundaries.
5. Incorporate probability calibration (SVC with probability=True) to output clinical confidence
scores rather than hard binary classifications.
```

Step 9.3: Next Version Retraining Strategy

- Defines the MLOps roadmap for retraining and re-evaluating candidate models before updating production serialized pickle artifacts.

```
print("Next version: retrain with Age + Fare features on the full dataset, then re-evaluate
before replacing svc_trained_model.pkl")
```

```
Next version: retrain with Age + Fare features on the full dataset, then re-evaluate before repla
cing svc_trained_model.pkl
```